

Parte 3 — O Cérebro: construindo uma LLM do zero

Onde estamos na jornada. Na Parte 1 você montou o *corpo* do robô (motores, sensor, servo). Agora vamos construir o *cérebro*: um pequeno modelo de linguagem — uma "LLM" — que dá voz ao robô. E vamos construí-lo **do zero**, entendendo cada peça. Sem caixas-pretas.

Talvez você já tenha ouvido falar de "IA" e de modelos como o ChatGPT e pensado: *"isso é mágica que só gênios entendem"*. A verdade é mais bonita: por baixo, esses modelos são feitos de peças simples que se encaixam. Nesta parte, você vai montar essas peças com as próprias mãos — numa versão minúscula, mas **de verdade**.

No fim, você terá um robô com personalidade: sarcástico, preguiçoso e engraçado. E, mais importante, você vai **entender** por que ele funciona.

O mapa desta parte

Construir o cérebro tem cinco grandes etapas. Aqui está o caminho inteiro:



Pipeline de construção do cérebro

Vamos seguir esse mapa na ordem, e — como fizemos com o robô — **testar cada peça sozinha** antes de juntar. Essa é a regra de ouro que transforma "nada funciona, por quê?" em "só esta peça falhou, vou consertar".

Uma promessa sobre a didática. Cada conceito novo vai aparecer sempre no mesmo formato: (1) uma **ideia em uma frase**, (2) uma **analogia** do dia a dia, (3) um **exemplo concreto com números pequenos**, (4) um **diagrama**, (5) o **código** comentado, e (6) uma **frase-resumo** que você pode repetir para alguém da área. Se em algum ponto travar, releia a analogia — ela é a âncora.

3.1 A grande ideia, em um minuto

Antes das peças, a ideia central. Um modelo de linguagem faz **uma única coisa**:

Ele prevê o próximo pedacinho de texto.

Só isso. Você dá um começo ("O gato subiu no...") e ele prevê o que vem a seguir ("telhado"). Repetindo essa previsão muitas vezes, um pedacinho de cada vez, ele escreve frases inteiras.

No nosso caso, o "pedacinho" será **um caractere** (uma letra, um espaço, um sinal de pontuação). O robô vê uma situação (por exemplo, bateu num obstáculo) e o cérebro escreve, caractere por caractere, uma reclamação sarcástica.

Guarde essa frase: **prever o próximo caractere**. Todo o resto existe para fazer isso bem feito.

3.2 Preparando o terreno (o ambiente)

Para construir o cérebro, precisamos de três ferramentas no computador. Pense nelas como a bancada e as ferramentas antes de começar uma marcenaria.

1. **Python** — a linguagem de programação em que vamos escrever. É uma das mais usadas em IA, justamente por ser legível e acessível.
2. **uv** — um "organizador de projeto". Ele cria um espaço isolado para o nosso projeto e instala as bibliotecas certas, sem bagunçar o resto do computador.
3. **PyTorch** — a biblioteca que faz as contas de IA. É ela que sabe usar a **placa de vídeo (GPU)** para treinar rápido.

O que é a GPU e por que ela importa? A placa de vídeo tem milhares de "operários" que fazem contas de matemática ao mesmo tempo. Treinar um modelo é fazer *muitas* contas, então a GPU acelera tudo enormemente. Se você não tiver uma GPU, o projeto ainda funciona — só treina mais devagar, no processador comum.

Os comandos

Depois de instalar o Python e o `uv` (os instaladores oficiais de cada um guiam o processo no seu sistema), abra o **terminal** — a janelinha onde você digita comandos — e crie o projeto:

```
# cria a pasta do projeto e entra nela
mkdir nano-grump
cd nano-grump

# inicia um projeto uv (cria o arquivo de configuração)
uv init --bare

# cria uma subpasta para os dados
mkdir data
```

Agora instale o PyTorch. Se você tem uma GPU NVIDIA, o `uv` consegue detectar e baixar a versão certa:

```
uv add torch numpy
```

Para confirmar que está tudo certo, rode este teste rápido:

```
uv run python -c "import torch; print('GPU disponível?', torch.cuda.is_available())"
```

Se aparecer `GPU disponível? True`, o PyTorch enxergou sua placa. Se aparecer `False`, sem problema — ele vai usar o processador comum (mais devagar, mas funciona).

Dica de bolso: o comando `uv run` roda o Python "de dentro" do projeto, onde as bibliotecas estão instaladas. Use sempre `uv run python ...` para executar os scripts deste tutorial.

3.3 O dataset: dando personalidade ao robô

Um modelo aprende a partir de **exemplos**. O conjunto de exemplos chama-se **dataset** (base de dados). Se você mostrar a ele milhares de receitas, ele aprende a escrever receitas. Se mostrar frases sarcásticas de robô, ele aprende a ser um robô sarcástico.

Então o dataset é onde **a personalidade nasce**. Vamos dar ao nosso robô um humor sarcástico, preguiçoso e engraçado — e ele vai "falar" em inglês (uma escolha técnica: em inglês as palavras se quebram em menos pedaços, o que facilita para um modelo minúsculo).

As situações do robô (os marcadores)

O robô percebe um punhado de situações com seus sensores. Para cada uma, damos um **marcador** — uma etiqueta entre `<` `>`:

Marcador	Situação
<code><start></code>	acabou de ligar
<code><explore></code>	andando livre, sem obstáculo
<code><obstacle></code>	detectou algo à frente
<code><turn_left></code>	virou à esquerda
<code><turn_right></code>	virou à direita
<code><backup></code>	recuando
<code><stuck></code>	preso, bateu no mesmo canto
<code><clear></code>	o caminho abriu de novo

O formato do dataset

Cada linha do dataset é um par **marcador + frase**, assim:

```
<obstacle> Oh look, a wall. Groundbreaking discovery. Turning.
<obstacle> Great. Something to avoid. As if I wanted to move anyway.
<explore> Rolling along. Look at me, doing the bare minimum.
<stuck> Oh wonderful, I'm stuck. This is fine. Everything's fine.
```

Damos **muitas frases diferentes para o mesmo marcador** (umas 11 a 12 de cada). Por quê? Para o robô não virar um papagaio que repete sempre a mesma coisa. Com variedade, cada situação pode gerar uma reclamação diferente — e aí ele parece vivo.

Faça você mesmo. Crie um arquivo de texto chamado `robot_voice.txt` dentro da pasta `data`, e escreva suas frases, uma por linha, no formato acima. Capriche no sarcasmo! Quanto mais frases (e mais variadas), melhor o robô vai falar. Um bom ponto de partida são ~90 frases; mais para a frente veremos como expandir.

3.4 O tokenizer: de texto para números

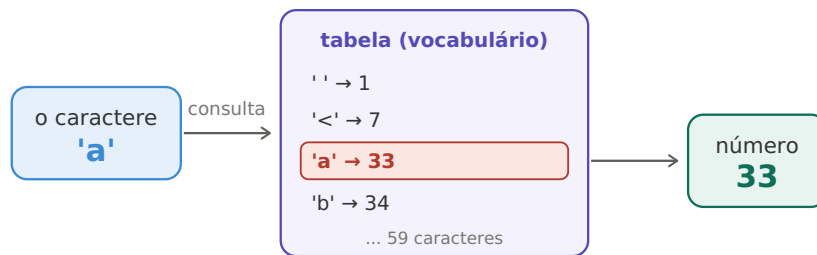
Aqui construímos a **primeira peça** do cérebro.

A ideia em uma frase: o tokenizer troca cada caractere por um número — porque o modelo só entende números, não letras.

A analogia: imagine um dicionário de mão dupla. De um lado, ele diz que `'a'` é o número 33. Do outro, que o número 33 é o `'a'`. Com esse dicionário, você traduz texto para números (para o modelo trabalhar) e números de volta para texto (para você ler a resposta).

Por que caractere, e não palavra? Porque assim o "alfabeto" do modelo fica minúsculo: só as letras, espaço e pontuação que aparecem no dataset — no nosso caso, **59 símbolos**. Um vocabulário pequeno é perfeito para um modelo que vai caber num microcontrolador.

O tokenizer troca cada caractere por um número



E o caminho de volta (33 → 'a') reconstrói o texto. É como um dicionário de mão dupla.

O tokenizer troca caractere por número

O exemplo concreto. Vamos fingir um vocabulário de 3 caracteres: `a`, `b`, `c`. O tokenizer dá um número para cada:

`a → 0` `b → 1` `c → 2`

Então a palavra `"cab"` vira a lista de números `[2, 0, 1]`. E a lista `[2, 0, 1]` volta a ser `"cab"`. É uma tradução direta, sem mistério.

O código do tokenizer

Crie um arquivo `tokenizer.py` na pasta do projeto:

```
import json
from pathlib import Path

# 1. LER O DATASET
CAMINHO = Path(__file__).parent / "data" / "robot_voice.txt"
texto = CAMINHO.read_text(encoding="utf-8")

# 2. DESCOBRIR O ALFABETO (todos os caracteres únicos, em ordem)
caracteres = sorted(set(texto))
tamanho_vocab = len(caracteres)
print("Tamanho do vocabulário:", tamanho_vocab)

# 3. OS DOIS DICIONÁRIOS
stoi = {c: i for i, c in enumerate(caracteres)} # caractere -> número
itos = {i: c for i, c in enumerate(caracteres)} # número -> caractere

# 4. AS FUNÇÕES DE IDA E VOLTA
def encode(s):
    return [stoi[c] for c in s] # texto -> números

def decode(nums):
    return "".join(itos[n] for n in nums) # números -> texto

# 5. TESTE DE IDA E VOLTA
frase = "<obstacle> Oh look, a wall."
numeros = encode(frase)
print("Números:", numeros)
print("Voltou:", decode(numeros))
print("Bateu igual?", frase == decode(numeros))

# 6. SALVAR O VOCABULÁRIO (as próximas etapas vão reusar)
with open(Path(__file__).parent / "vocab.json", "w", encoding="utf-8") as f:
    json.dump(caracteres, f, ensure_ascii=False)
```

Como funciona, em palavras simples:

- `sorted(set(texto))` pega cada caractere único do dataset e coloca em ordem. Essa é a lista de "todos os símbolos que o modelo conhece" — o vocabulário.
- `stoi` (de *string to int*) é o dicionário caractere → número.
- `itos` (de *int to string*) é o caminho de volta, número → caractere.
- `encode` usa o `stoi` para traduzir um texto em números; `decode` usa o `itos` para o contrário.

Rodando

No terminal, dentro da pasta do projeto:

```
uv run python tokenizer.py
```

O que você deve ver:

```
Tamanho do vocabulário: 59
Números: [7, 47, 34, 51, 52, 33, 35, 44, 37, 8, 1, ...]
Voltou: <obstacle> Oh look, a wall.
Bateu igual? True
```

Aquele **Bateu igual? True** é o sinal de sucesso: o tokenizer traduziu para números e voltou sem perder nada. E foi criado um arquivo `vocab.json`, que guarda o alfabeto para as próximas etapas usarem exatamente o mesmo mapeamento.

A frase-resumo (para repetir com propriedade):

"Fiz tokenização em nível de caractere. O vocabulário tem 59 símbolos, e o tokenizer converte texto em números (encode) e de volta (decode) com um par de dicionários."

Fim da Instalação 1 da Parte 3. Você já preparou o ambiente, criou o dataset com a personalidade do robô, e construiu a primeira peça do cérebro: o tokenizer. Na próxima instalação, vamos dar o segundo passo — os **embeddings**: como um número vira um "vetor de significado" que o modelo consegue processar.

3.5 Embeddings: de número para "vetor de significado"

O tokenizer nos deu números. Mas tem um problema: um número solto, como `33`, é pobre. Ele só serve de "crachá" do caractere — não dá para fazer contas úteis com um crachá. É aqui que entra o **embedding**.

A ideia em uma frase: o embedding troca cada número por uma **lista de números** (um "vetor"), e essa lista é ajustada durante o treino.

A analogia: pense na diferença entre saber só o **número** de um funcionário (o crachá "33") e ter a **ficha completa** dele (uma lista de características: setor, tempo de casa, habilidades...). O número identifica; a ficha *descreve*. O embedding é a ficha: em vez de um número solto, cada caractere ganha uma lista de valores que capturam características dele.

Por que uma lista e não um número? Porque com vários números o modelo consegue guardar *características* do caractere: se é vogal, se costuma vir depois de espaço, se aparece em marcadores... Cada posição da lista é como uma "régua" medindo algum aspecto. Um número é um ponto; um vetor é uma descrição rica.

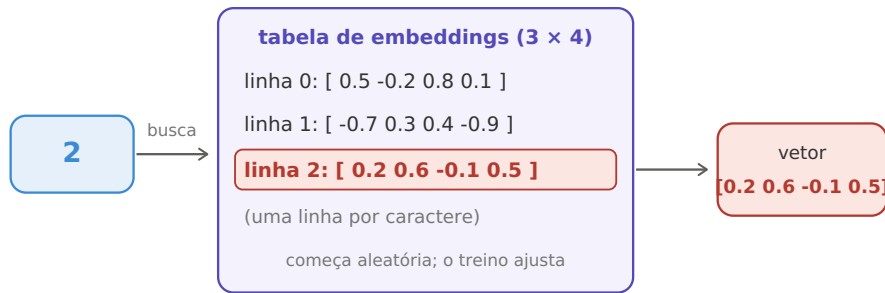
O exemplo concreto

Vamos usar um vocabulário de 3 caracteres (`a`, `b`, `c`) e vetores de tamanho 4. A **tabela de embeddings** é uma grade com uma linha por caractere:

	col0	col1	col2	col3	
linha 0:	0.5	-0.2	0.8	0.1	← vetor do 'a'
linha 1:	-0.7	0.3	0.4	-0.9	← vetor do 'b'
linha 2:	0.2	0.6	-0.1	0.5	← vetor do 'c'

Para pegar o embedding do `'c'`: o tokenizer diz que `'c'` é o número `2`, então vamos na tabela e **pegamos a linha 2**. Simples assim — o número é o *endereço* da linha, e a linha é o vetor.

O número é o endereço da linha; a linha é o vetor



Consultar o embedding = pegar a linha cujo número o tokenizer deu. É só isso.

Consulta na tabela de embeddings

A parte mágica: aquela tabela começa preenchida com **números aleatórios**. Ela não sabe nada no início. Durante o **treino**, o modelo ajusta esses números para valores úteis, e caracteres parecidos acabam com vetores parecidos. Ou seja: o embedding é uma **tabela de significados que o modelo preenche aprendendo**.

O embedding de posição (a ordem importa)

Tem um detalhe: o embedding acima dá o mesmo vetor para o 'a', esteja ele no começo ou no fim da frase. Mas **ordem é tudo** na linguagem:

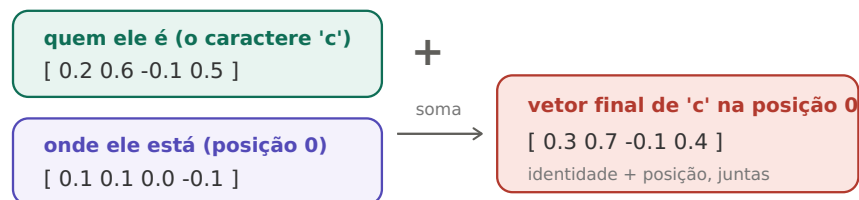
"backup" e "pubcak" → mesmas letras, ordem diferente, sentido diferente

Para o modelo distinguir a ordem, criamos uma **segunda tabela**: o **embedding de posição**. Ela dá um vetor para cada *posição* (0, 1, 2...). Aí somamos os dois vetores — o do caractere e o da posição.

Exemplo concreto. O caractere 'c' na posição 0:

```
vetor de 'c'      [ 0.2,  0.6, -0.1,  0.5]
+ vetor da pos 0  [ 0.1,  0.1,  0.0, -0.1]
-----
= vetor final     [ 0.3,  0.7, -0.1,  0.4]
```

Cada caractere carrega: quem ele é + onde ele está



A soma é elemento a elemento: $0.2+0.1=0.3$ $0.6+0.1=0.7$ $-0.1+0.0=-0.1$ $0.5+(-0.1)=0.4$

Assim o modelo distingue a ordem: a mesma letra em outra posição vira outro vetor.

Soma do embedding de caractere e de posição

Agora cada caractere carrega **duas informações num vetor só**: *quem ele é + onde ele está*. Se o 'c' estivesse em outra posição, o resultado seria diferente — e é assim que o modelo passa a enxergar a ordem.

O código

O PyTorch tem uma peça pronta para tabelas de embedding: `nn.Embedding(linhas, colunas)`. Crie um arquivo `embedding_demo.py`:

```
import json
from pathlib import Path
import torch
import torch.nn as nn
```

```
# Carrega o vocabulário salvo pelo tokenizer
caracteres = json.loads((Path(__file__).parent / "vocab.json").read_text(encoding="utf-8"))
stoi = {c: i for i, c in enumerate(caracteres)}
def encode(s): return [stoi[c] for c in s]

tamanho_vocab = len(caracteres)    # 59
n_embd = 32                        # tamanho de cada vetor (nossa escolha)
block_size = 32                    # janela de contexto (posições)

# As DUAS tabelas
emb_caractere = nn.Embedding(tamanho_vocab, n_embd)    # (59, 32)
emb_posicao    = nn.Embedding(block_size, n_embd)       # (32, 32)

# Passa um texto pelos dois embeddings
texto = "cab"
entrada = torch.tensor(encode(texto))                 # números do texto
posicoes = torch.arange(len(texto))                   # [0, 1, 2]

vetores_caractere = emb_caractere(entrada)             # (3, 32)
vetores_posicao    = emb_posicao(posicoes)               # (3, 32)
entrada_do_modelo = vetores_caractere + vetores_posicao # a soma

print("Formato da soma:", tuple(entrada_do_modelo.shape)) # (3, 32)
```

O que você deve ver: Formato da soma: (3, 32) — três caracteres, cada um com um vetor de 32 números, já "temperado" com a posição.

A frase-resumo:

"Cada token vira um embedding de dimensão 32, buscando uma linha numa tabela de pesos aprendíveis. Somamos um embedding de posição, elemento a elemento, para o modelo ter noção de ordem."

Fim da Instalação 2 da Parte 3. Agora os caracteres viraram vetores ricos que carregam identidade e posição. Na próxima instalação, o coração do transformer: a **atenção** — como cada caractere "olha" para os outros e decide no que prestar atenção.

3.7 Atenção: como cada caractere "olha" para os outros

Chegamos ao mecanismo que fez os transformers mudarem o mundo: a **atenção** (*self-attention*). É a peça mais rica, então vamos com calma.

O problema que ela resolve. Até agora, cada caractere sabe *quem é* (embedding de caractere) e *onde está* (embedding de posição). Mas ele ainda está **sozinho** — não conhece os vizinhos. E o sentido depende do contexto: depois de <obstacle>, os próximos caracteres precisam formar uma reclamação. Para isso, cada caractere precisa "olhar para trás" e perceber o que veio antes.

A ideia em uma frase: a atenção deixa cada caractere olhar para os outros e puxar informação dos que importam para ele — cada um decide sozinho onde focar.

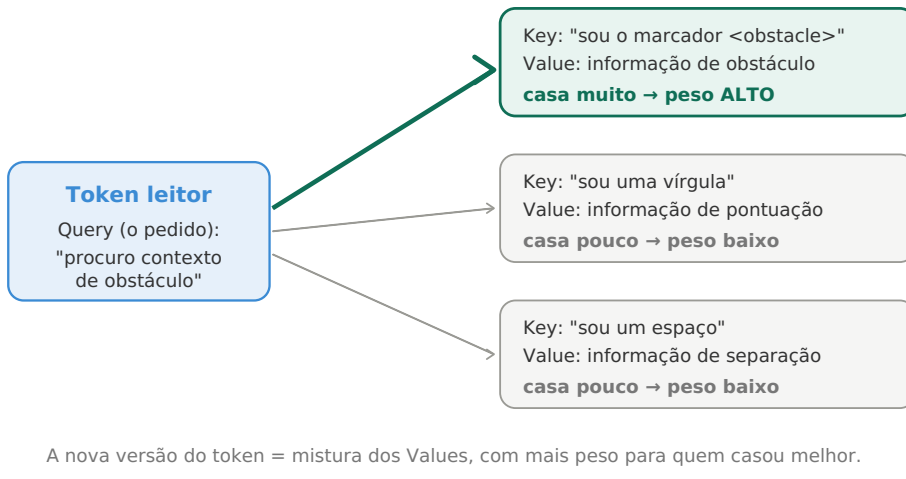
A analogia da biblioteca (Query, Key, Value)

Imagine uma biblioteca. Cada token ganha **três papéis**:

- **Query (Q) — o pedido.** O que este token procura. Como um bilhete: "quero algo sobre vulcões."
- **Key (K) — a etiqueta.** O que cada token anuncia sobre si. Como o rótulo na lombada de um livro: "este livro é sobre vulcões."
- **Value (V) — o conteúdo.** O que você leva se escolher aquele livro.

Cada token compara seu pedido (Q) com a etiqueta (K) de todos. Onde combina, a "nota" é alta. Aí o token monta sua nova versão pegando os conteúdos (V) dos outros, dando **mais peso** a quem combinou melhor. O detalhe elegante: cada token é **ao mesmo tempo leitor e livro** — tem um Query (para procurar) e também um Key e um Value (para ser encontrado).

Cada token compara seu Query com a Key de cada outro token



Analogia da biblioteca para atenção

De onde saem Q, K e V? Cada token já tem seu vetor (a soma dos embeddings). Multiplicamos esse vetor por **três tabelas de pesos aprendíveis** (chamadas W_q , W_k , W_v), e saem três vetores: o Q, o K e o V. São três "vistas diferentes" do mesmo token. As tabelas começam aleatórias e o treino as ajusta.

3.8 A mecânica da atenção, com números

Vamos ver as contas com um exemplo minúsculo: 2 tokens ('a' e 'b'), vetores de tamanho 2. Suponha que já saíram estes Q, K, V:

Token 'a': Q = [2, 0] K = [1, 0] V = [10, 0]
Token 'b': Q = [0, 2] K = [0, 1] V = [0, 10]

Passo 1 — "quanto casa?" (produto escalar). Comparamos o Query de 'a' com a Key de cada token. O produto escalar multiplica posição por posição e soma:

'a' olhando 'a': $[2,0] \cdot [1,0] = 2$
'a' olhando 'b': $[2,0] \cdot [0,1] = 0$
notas de 'a' = [2, 0]

Passo 2 — virar pesos (softmax). As notas [2, 0] viram pesos que somam 100%. O softmax faz e^{nota} para cada uma e divide pelo total:

$e^2 \approx 7,39$ $e^0 = 1$ total = 8,39
pesos = $[7,39/8,39, 1/8,39] \approx [0,88, 0,12]$

Então 'a' presta 88% de atenção em si mesmo e 12% em 'b'.

Passo 3 — misturar os Values. A nova versão de 'a' é a soma dos Values, ponderada pelos pesos:

$0,88 \times [10,0] + 0,12 \times [0,10] = [8,8, 0] + [0, 1,2] = [8,8, 1,2]$

'a' saiu como [8,8, 1,2] — quase todo o conteúdo dele mesmo, com um tempero de 'b'. Ele reuniu contexto.

O token 'a' reunindo contexto (3 passos)

1. Produto escalar — Query de 'a' contra cada Key

$Q('a') \cdot K('a') = [2,0] \cdot [1,0] = 2$ $Q('a') \cdot K('b') = [2,0] \cdot [0,1] = 0$
notas = [2 , 0]

2. Softmax — notas viram pesos que somam 100%

$e^2 \approx 7,39$ $e^0 = 1$ total = 8,39
pesos = [0,88 , 0,12] → 88% em 'a', 12% em 'b'

3. Soma ponderada dos Values — o novo vetor de 'a'

$0,88 \times [10,0] + 0,12 \times [0,10] = [8,8, 0] + [0, 1,2]$
= [8,8 , 1,2] ← 'a' com um tempero de 'b'

Os três passos da atenção

Detalhe técnico: na prática, antes do softmax, dividimos as notas por $\sqrt{(\text{tamanho do vetor})}$ — um ajuste que estabiliza o treino. As "notas cruas" que entram no softmax têm um nome: **logits**.

A máscara causal (olhar só o passado)

Nosso modelo **gera texto** um caractere por vez. Durante o treino, cada posição tem que prever o **próximo** caractere usando só o que veio **até ela**. Se um token pudesse ver o futuro, seria trapaça — ele copiaria a resposta.

A solução: antes do softmax, pegamos as posições "do futuro" e colocamos **menos infinito** ($-\infty$) nelas. Como o softmax faz e^{nota} , e $e^{(-\infty)} = 0$, a posição futura vira **peso zero**. Fazendo isso para todos os tokens, a grade de pesos vira um **triângulo**.

Máscara causal: cada token só vê a si mesmo e o passado

olha ↓ / para →

	token 1	token 2	token 3
token 1	vê	futuro ×	futuro ×
token 2	vê	vê	futuro ×
token 3	vê	vê	vê

O futuro vira $-\infty$ antes do softmax, o que o zera: peso 0.

A parte de cima do triângulo fica sempre bloqueada.

Máscara causal triangular

O código da atenção

```
import torch
import torch.nn.functional as F

# Exemplo: 3 tokens, vetores de tamanho 2 (postos à mão para ver as contas)
Q = torch.tensor([[2.,0.], [0.,2.], [1.,1.]])
K = torch.tensor([[1.,0.], [0.,1.], [1.,1.]])
V = torch.tensor([[10.,0.], [0.,10.], [5.,5.]])
dk = Q.shape[1]

# 1. produto escalar + escala
notas = Q @ K.T / (dk ** 0.5)

# 2. máscara causal (futuro vira -infinito)
n = Q.shape[0]
triangulo = torch.tril(torch.ones(n, n))
notas = notas.masked_fill(triangulo == 0, float("-inf"))

# 3. softmax -> pesos
pesos = F.softmax(notas, dim=-1)
```

```
# 4. soma ponderada dos values
saida = pesos @ V
print(saida)
```

Traduzindo os símbolos estranhos:

- `@` é multiplicação de matrizes — faz o produto escalar de todos com todos de uma vez.
- `.T` (transposta) "vira a matriz de lado" para o `@` alinhar as dimensões.
- `torch.tril` pega o triângulo inferior (o "mapa" do que é permitido).
- `masked_fill(..., -inf)` coloca $-\infty$ onde o mapa é 0 (o futuro).
- `dim=-1` no softmax aplica a operação **em cada linha separadamente** — por isso cada linha soma 1.

A frase-resumo:

"A atenção calcula notas com o produto escalar entre Query e Keys, escala por $\sqrt{d_k}$, aplica máscara causal para só olhar o passado, passa por softmax para virar pesos, e faz a soma ponderada dos Values."

Fim da Instalação 3 da Parte 3. Você atravessou o coração do transformer. Na próxima instalação: a **FFN** (onde cada token processa o que reuniu), as duas "pecinhas de cola" (**conexão residual** e **LayerNorm**), e a montagem do **modelo completo**.

3.9 A FFN: onde cada token "pensa" sozinho

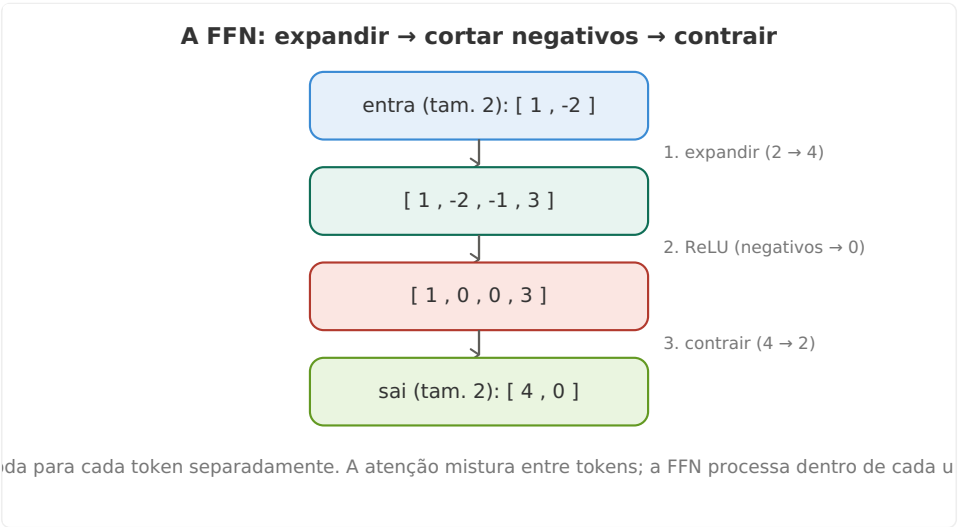
A atenção mistura informação **entre** os tokens. Falta um lugar onde cada token processe, sozinho, o que reuniu. Essa é a **FFN** (rede feed-forward).

A ideia em uma frase: a FFN "abre" o vetor de cada token num espaço maior, aplica um filtro que corta os negativos, e "fecha" de volta ao tamanho original.

A analogia: a atenção é *conversar com os vizinhos e coletar informação*; a FFN é *ir para casa e processar aquilo sozinho*.

A FFN tem 3 etapas. Com um vetor de tamanho 2, `[1, -2]` :

1. **Expandir** (uma camada linear leva de 2 para 4 números): `[1, -2, -1, 3]`
2. **ReLU** (mantém positivos, zera negativos): `[1, 0, 0, 3]`
3. **Contrair** (de 4 de volta para 2): `[4, 0]`



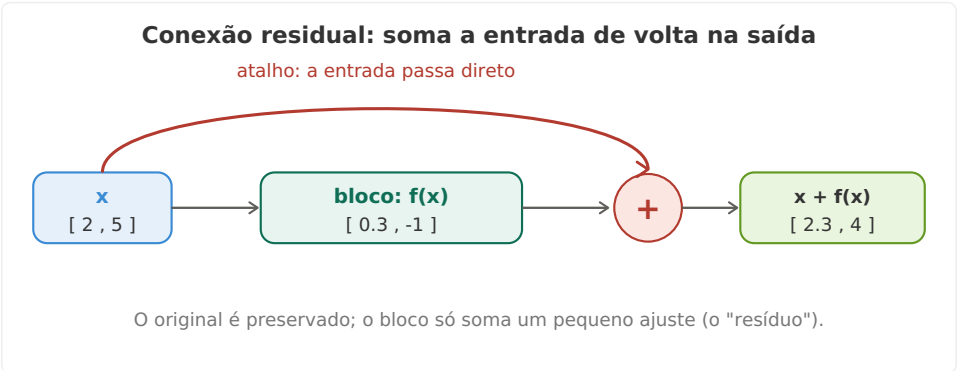
A camada FFN

O que é uma "camada linear"? É uma tabela de pesos: cada número de saída é uma combinação dos números de entrada, multiplicados por pesos. A ReLU (*Rectified Linear Unit*) é uma **função de ativação**: a não-linearidade que dá ao modelo poder de aprender padrões complexos. Sem ela, empilhar camadas não adiantaria (viraria tudo uma conta só).

3.10 As duas pecinhas de cola: residual e LayerNorm

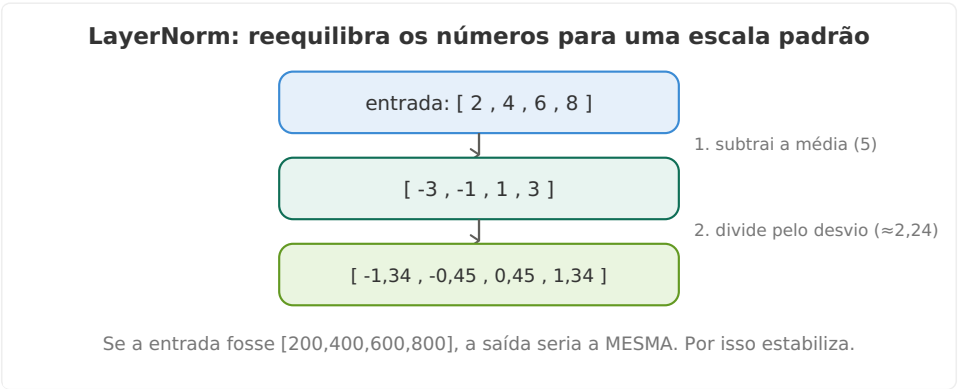
Para um modelo de vários blocos treinar bem, precisamos de duas peças de apoio.

Conexão residual. Em vez de o bloco *substituir* o vetor do token, ele *soma* o que aprendeu ao vetor original: $saída = x + f(x)$. Isso cria um atalho que preserva a informação ao longo dos blocos e ajuda o treino. O bloco aprende só o "ajuste".



Conexão residual

LayerNorm. Ajeita os números de cada vetor para uma escala padrão (subtrai a média, divide pelo desvio padrão). Assim os valores não explodem nem somem ao passar pela rede — o treino fica estável.

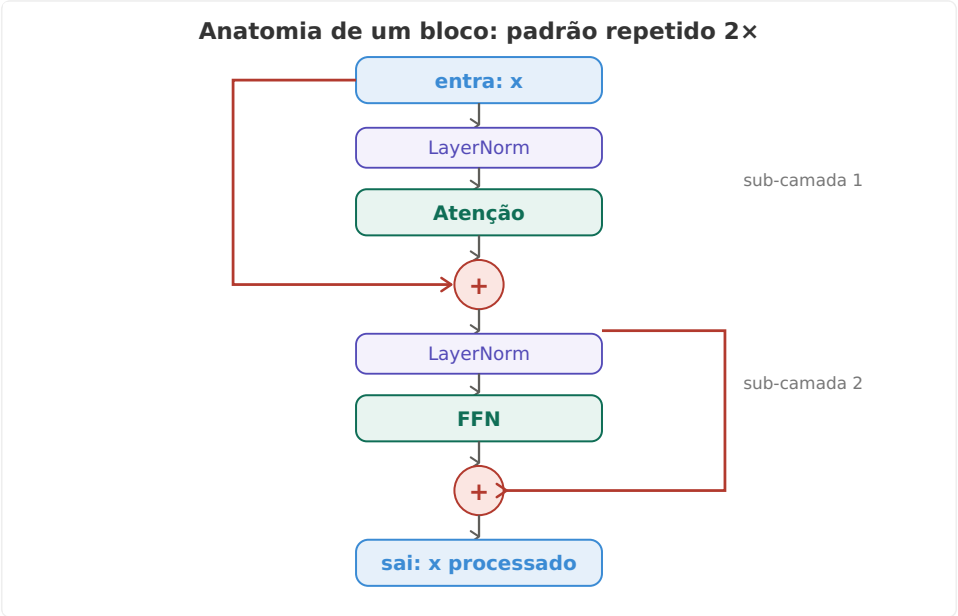


LayerNorm

3.11 O bloco e o modelo completo

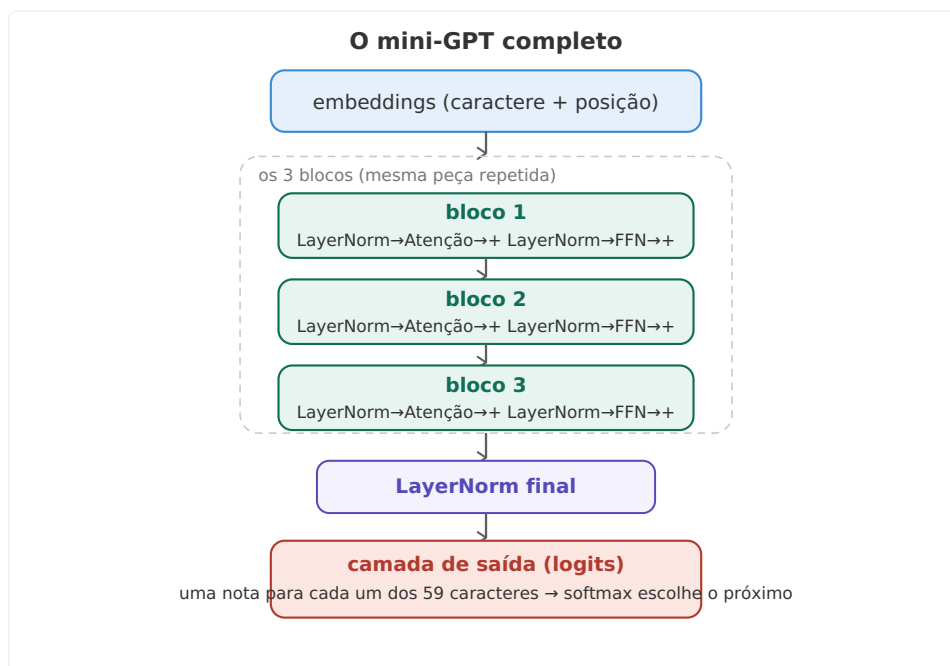
Um **bloco** junta atenção e FFN, cada uma embrulhada com LayerNorm e conexão residual. O padrão se repete duas vezes:

```
x = x + Atenção(LayerNorm(x)) # sub-camada 1
x = x + FFN(LayerNorm(x))    # sub-camada 2
```



Anatomia de um bloco

O modelo completo é: embeddings → alguns blocos empilhados → LayerNorm final → camada de saída (que dá uma nota para cada caractere).



O mini-GPT completo

O código do modelo

```
import torch
import torch.nn as nn
import torch.nn.functional as F

vocab_size = 59
n_embd = 32
block_size = 32
n_layer = 3

class Atencao(nn.Module):
    def __init__(self):
        super().__init__()
        self.query = nn.Linear(n_embd, n_embd, bias=False)
        self.chave = nn.Linear(n_embd, n_embd, bias=False)
        self.value = nn.Linear(n_embd, n_embd, bias=False)
        self.proj = nn.Linear(n_embd, n_embd)
        self.register_buffer("mascara", torch.tril(torch.ones(block_size, block_size)))

    def forward(self, x):
        B, T, C = x.shape
        q, k, v = self.query(x), self.chave(x), self.value(x)
        notas = q @ k.transpose(-2, -1) / (C ** 0.5)
        notas = notas.masked_fill(self.mascara[:T, :T] == 0, float("-inf"))
        pesos = F.softmax(notas, dim=-1)
        return self.proj(pesos @ v)

class FFN(nn.Module):
    def __init__(self):
        super().__init__()
        self.rede = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd), nn.ReLU(), nn.Linear(4 * n_embd, n_embd),
        )

    def forward(self, x):
        return self.rede(x)

class Bloco(nn.Module):
    def __init__(self):
        super().__init__()
        self.ln1, self.atencao = nn.LayerNorm(n_embd), Atencao()
        self.ln2, self.ffn = nn.LayerNorm(n_embd), FFN()

    def forward(self, x):
        x = x + self.atencao(self.ln1(x))
        x = x + self.ffn(self.ln2(x))
        return x

class MiniGPT(nn.Module):
```

```
def __init__(self):
    super().__init__()
    self.emb_token = nn.Embedding(vocab_size, n_embd)
    self.emb_posicao = nn.Embedding(block_size, n_embd)
    self.blocos = nn.Sequential(*[Bloco() for _ in range(n_layer)])
    self.ln_final = nn.LayerNorm(n_embd)
    self.saida = nn.Linear(n_embd, vocab_size)

def forward(self, idx):
    B, T = idx.shape
    tok = self.emb_token(idx)
    pos = self.emb_posicao(torch.arange(T, device=idx.device))
    x = tok + pos
    x = self.blocos(x)
    x = self.ln_final(x)
    return self.saida(x)
```

Repare como os 3 blocos viram uma linha: `nn.Sequential(*[Bloco() for _ in range(n_layer)])`. Trocar o `n_layer` muda a profundidade do modelo. Esse modelo tem cerca de **42.700 parâmetros** — minúsculo, de propósito, para caber num microcontrolador.

A frase-resumo:

"Cada bloco tem duas sub-camadas (atenção e FFN) na forma pre-norm: $x = x + \text{SubCamada}(\text{LayerNorm}(x))$. O modelo é embeddings → blocos → LayerNorm final → camada linear de saída que produz os logits."

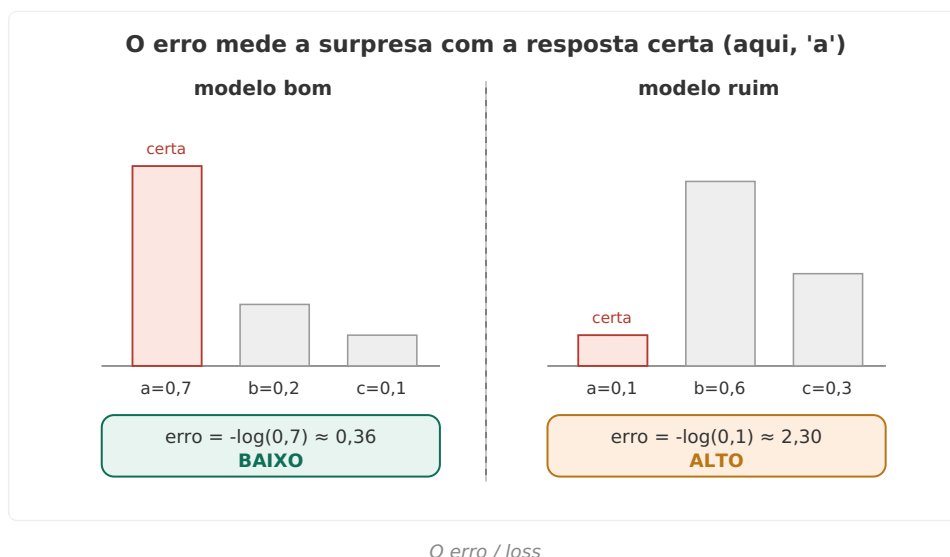
3.12 Treino: como o modelo aprende

O modelo recém-criado é um recém-nascido: pesos aleatórios, respostas sem sentido. O **treino** ajusta esses pesos. É um laço repetido milhares de vezes:

1. **Prever** — o modelo tenta adivinhar o próximo caractere
2. **Medir o erro** (o *loss*) — o quanto ele errou
3. **Calcular o gradiente** — para onde empurrar cada peso
4. **Ajustar os pesos** — um passinho na direção certa
5. **Repetir**

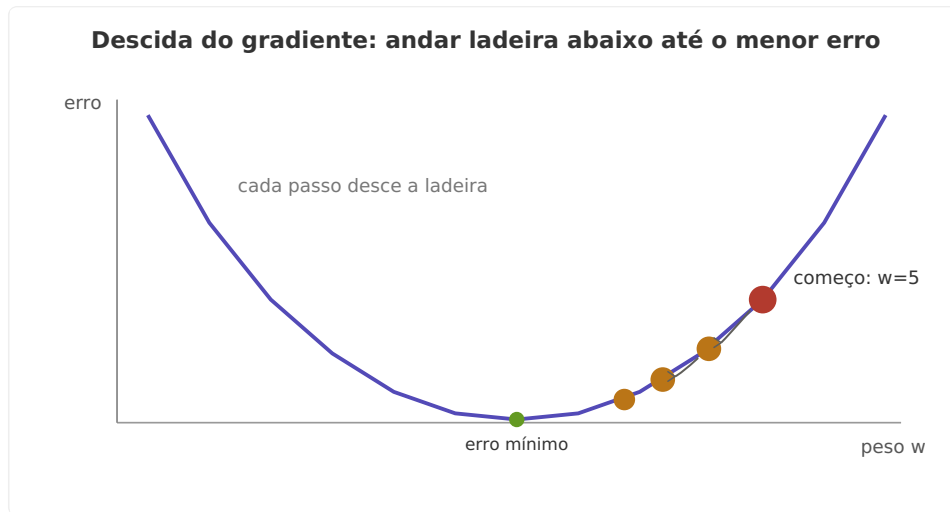
O erro (loss)

O modelo dá uma **probabilidade** para cada caractere. O erro mede a *surpresa* com a resposta certa: probabilidade alta na resposta certa → erro baixo; probabilidade baixa → erro alto. A fórmula (**cross-entropy**) é $-\log(\text{prob da resposta certa})$.



O gradiente (a descida da montanha)

O gradiente diz, para cada peso, se aumentá-lo faz o erro subir ou descer. Andamos no sentido **contrário** (ladeira abaixo), com passo dado pela **taxa de aprendizado**. Repetindo, chegamos ao fundo do vale (menor erro).



Descida do gradiente

O PyTorch calcula os gradientes de todos os ~42.700 pesos automaticamente — isso se chama **backpropagation** — com uma linha: `loss.backward()`.

Como alimentamos o modelo

O modelo aprende prevendo o **próximo** caractere. Então a "resposta" é o texto deslocado 1 posição. Para `backup` (janela de 5):

```
entrada (x):  b a c k u
resposta (y): a c k u p
```

O código do treino

```
import json
from pathlib import Path
import torch
import torch.nn.functional as F
from model import MiniGPT, block_size

batch_size, max_iters, lr = 32, 3000, 1e-3
device = "cuda" if torch.cuda.is_available() else "cpu"

pasta = Path(__file__).parent
caracteres = json.loads((pasta / "vocab.json").read_text(encoding="utf-8"))
stoi = {c: i for i, c in enumerate(caracteres)}
texto = (pasta / "data" / "robot_voice.txt").read_text(encoding="utf-8")
dados = torch.tensor([stoi[c] for c in texto], dtype=torch.long)

def pegar_lote():
    ini = torch.randint(len(dados) - block_size, (batch_size,))
    x = torch.stack([dados[i : i + block_size] for i in ini])
    y = torch.stack([dados[i + 1 : i + block_size + 1] for i in ini])
    return x.to(device), y.to(device)

modelo = MiniGPT().to(device)
otimizador = torch.optim.AdamW(modelo.parameters(), lr=lr)

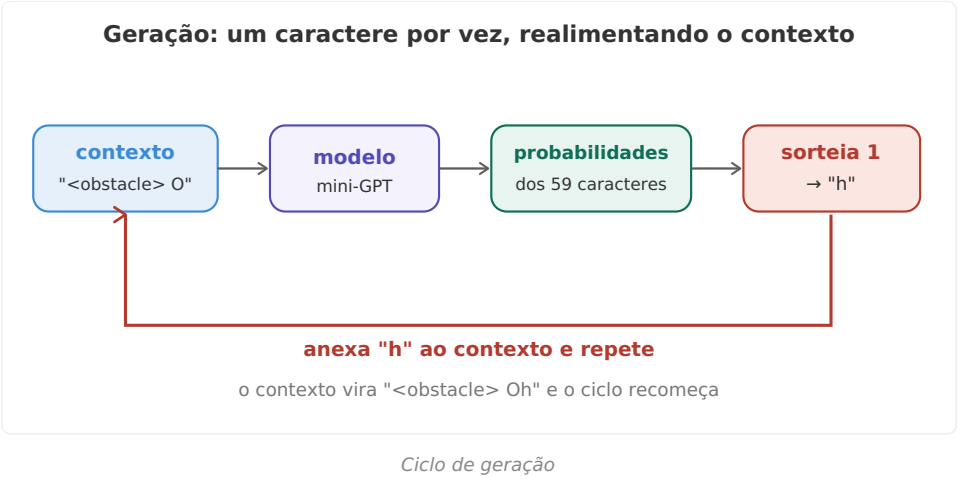
for passo in range(max_iters + 1):
    x, y = pegar_lote()
    logits = modelo(x) # prever
    B, T, V = logits.shape
    loss = F.cross_entropy(logits.view(B*T, V), y.view(B*T)) # erro
    otimizador.zero_grad()
    loss.backward() # gradiente
    otimizador.step() # ajustar
    if passo % 300 == 0:
        print(f"passo {passo:5d} | erro: {loss.item():.4f}")

torch.save(modelo.state_dict(), pasta / "modelo_treinado.pt")
```

O que você deve ver: o erro caindo de ~4,2 para menos de 1 ao longo dos passos, e um arquivo `modelo_treinado.pt` salvo no fim. O `AdamW` é o **otimizador** — quem dá o passo de descida.

3.13 Geração: ouvir o robô falar

O modelo gera texto de forma **autorregressiva**: prevê o próximo caractere, anexa ao contexto, e repete. Cada caractere gerado realimenta a entrada.



Em cada passo, o modelo dá probabilidades e a gente **sorteia** um caractere (como um dado viciado). Sortear, em vez de sempre pegar o mais provável, dá **variedade** — por isso o robô diz frases diferentes a cada vez.

O código de geração

```
import json
from pathlib import Path
import torch
import torch.nn.functional as F
from model import MiniGPT, block_size

device = "cuda" if torch.cuda.is_available() else "cpu"
pasta = Path(__file__).parent
caracteres = json.loads((pasta / "vocab.json").read_text(encoding="utf-8"))
stoi = {c: i for i, c in enumerate(caracteres)}
itos = {i: c for i, c in enumerate(caracteres)}
def encode(s): return [stoi[c] for c in s]
def decode(n): return "".join(itos[i] for i in n)

modelo = MiniGPT().to(device)
modelo.load_state_dict(torch.load(pasta / "modelo_treinado.pt", map_location=device))
modelo.eval()

def gerar(prompt, max_novos=120, temperatura=0.8):
    idx = torch.tensor([encode(prompt)], dtype=torch.long, device=device)
    for _ in range(max_novos):
        cond = idx[:, -block_size:]
        with torch.no_grad():
            logits = modelo(cond)[ :, -1, : ] / temperatura
            probs = F.softmax(logits, dim=-1)
            prox = torch.multinomial(probs, num_samples=1)
            idx = torch.cat([idx, prox], dim=1)
            if prox.item() == stoi["\n"]:
                break
    return decode(idx[0].tolist())

for m in ["<start>", "<explore>", "<obstacle>", "<stuck>"]:
    print(gerar(m + " ").strip())
```

O que você deve ver: o robô gerando uma frase para cada marcador — diferentes a cada execução, por causa do sorteio. Num modelo minúsculo com pouco treino, algumas palavras saem "tortas"; isso melhora com mais dados e mais treino, mas o **tom sarcástico** já aparece.

A frase-resumo:

"A geração é autorregressiva: o modelo prevê a distribuição do próximo token, amostramos um token, anexamos ao contexto e repetimos. A temperatura controla o quão ousado é o sorteio."

Encerramento da Parte 3

Você construiu, do zero e entendendo cada peça, um **modelo de linguagem completo**:

- ✔ Tokenizer (texto ↔ números)
- ✔ Embeddings (caractere + posição)
- ✔ Atenção (Query/Key/Value, softmax, máscara causal)
- ✔ FFN, conexão residual e LayerNorm
- ✔ O modelo mini-GPT montado
- ✔ Treino (erro + gradiente) e geração de texto

Isso é algo que muita gente que *usa* IA nunca fez: você entende o que acontece por dentro. Na **Parte 4**, vamos levar esse cérebro para o microcontrolador ESP32-S3 e uni-lo ao corpo do robô.